1.2em

Chapter 1

Methodology

Notation

The following symbols are used consistently throughout this chapter. Bold lower-case letters denote vectors; calligraphic upper-case letters denote sets or modules; θ always refers to the language model weights.

Symbol	Definition
q	Natural-language query (input to CAEM)
$\phi(q)$	Sentence-BERT embedding of q
e^*	Nearest episode retrieved from memory $\mathcal{M}$
s	FAISS cosine similarity between $\phi(q)$ and $\phi(e^*)$
r	Combined routing score: $r = 0.7\,s + 0.3\,\hat{u}_{\text{stored}}(e^*)$
u_{pre}	Pre-generation confidence (encoder-only, Stage 3)
C_{conv}	Conversational coherence signal (Stage 3)
$u_{\text{MC-pre}}$	MC-dropout confidence before generation
$\hat{u}$	Post-generation confidence (Tier 2, Stage 4a)
$\hat{u}_{\text{stored}}$	Stored quality score assigned at verification (Stage 5)
c	Chain-of-thought reasoning produced by the model
a	Final answer produced by the model
p	RAG-retrieved passage (Tier 3 only)
$\mathcal{M}$	Episodic memory (FAISS index + metadata store)
K	Number of answer samples for semantic entropy ($K = 10$)
M	Number of reasoning chains for NLI/SC layers ($M = 3$)
θ	Language model weights (Phi-2 base)
θ_{prev}	Weights before the current self-improvement cycle
λ	L2 regularisation coefficient ($\lambda = 0.01$)
$\rho_{\min}$	Forgetting abort threshold ($\rho_{\min} = 0.93$)
p_{entail}	Mean NLI entailment probability (verification Layer 1)
s_{avg}	Mean pairwise chain cosine similarity (Layer 2)
h_{norm}	Normalised semantic entropy (Layer 3)
p_n	Data purity of memory at cycle n
α	True-positive rate of the verifier
P	Probability query is from the target distribution
T	Temperature parameter for sampling
ECE	Expected Calibration Error (M-bin, $M = 10$)
Hyperparameter Origin Tags	
[LIT]	Fixed from literature (requires citation)
[DES]	Design choice (principled starting value, e.g., thresholds)
[CAL]	Empirically calibrated (derived from calibration data)

1.1 System Architecture Overview

CAEM is not a post-hoc hallucination detector. It is an architectural intervention that prevents unreliable knowledge from accumulating in memory and uses verified knowledge to progressively improve the quality of future generation. The distinction matters: post-hoc detectors identify errors after a model has already committed to an answer [**?**, **?**]; CAEM changes what the model learns across cycles so that fewer errors are generated in the first place.

The motivation for an architectural approach comes directly from a finding in the benchmark literature. Lin et al. [**?**] demonstrated an *inverse scaling effect* on TruthfulQA: larger GPT-3 models scored *worse* than smaller ones, achieving only 21–25% on the combined truthful-and-informative metric against 94% human performance. Larger models more faithfully reproduce internet misconceptions present in their training data. Scaling alone does not solve hallucination and, for some categories, makes it worse. This result directly motivates a closed-loop architectural approach: if the model's parametric knowledge is the source of systematic errors, the solution is to reshape what the model learns, not simply to scale it up.

The Eight-Stage Pipeline

CAEM processes every query through a fixed eight-stage pipeline, grouped into three functional phases: *routing & generation* (Stages 1–4a), *verification & storage* (Stages 5–7), and *cyclic improvement* (Stage 8).

Stage	Name	Phase	Description
1	Input	*Routing & Gen.*	Accept a natural language question q.
2	Encode	*Routing & Gen.*	Compute SBERT embedding $\phi(q)$ (768-d, `all-mpnet-base-v2`). Retrieve nearest episode from memory; obtain similarity s and $\hat{u}_{\text{stored}}$.
3	Pre-routing confidence	*Routing & Gen.*	Compute lightweight u_{pre} from encoder-only signals (Section **??**). No generation at this stage.
4	Route	*Routing & Gen.*	Assign query to Tier 1, 2, or 3 via routing score and OR-condition safety veto (Section **??**).
4a	Post-gen. gate	*Routing & Gen.*	*Tier 2 only.* Compute $\hat{u}$ from four signals; accept if $\hat{u} \geq 0.6$, else escalate to Tier 3 (Section **??**).
5	Verify	*Verification & Storage*	Run NLI + self-consistency + semantic entropy pipeline (Section **??**). Tier 1 skips per-query re-verification; freshness maintained by retroactive re-verification (Section **??**).
6	Output	*Verification & Storage*	Return the verified answer a to the user.
7	Store	*Verification & Storage*	If accepted, compute $\hat{u}_{\text{stored}}$ and persist the episode in memory (Section **??**).
8	Self-improve	*Cyclic Improvement*	Fine-tune θ on all verified episodes from the current cycle; apply forgetting abort guard and retroactive re-verification (Section **??**).

Table 1.1: CAEM eight-stage pipeline overview.

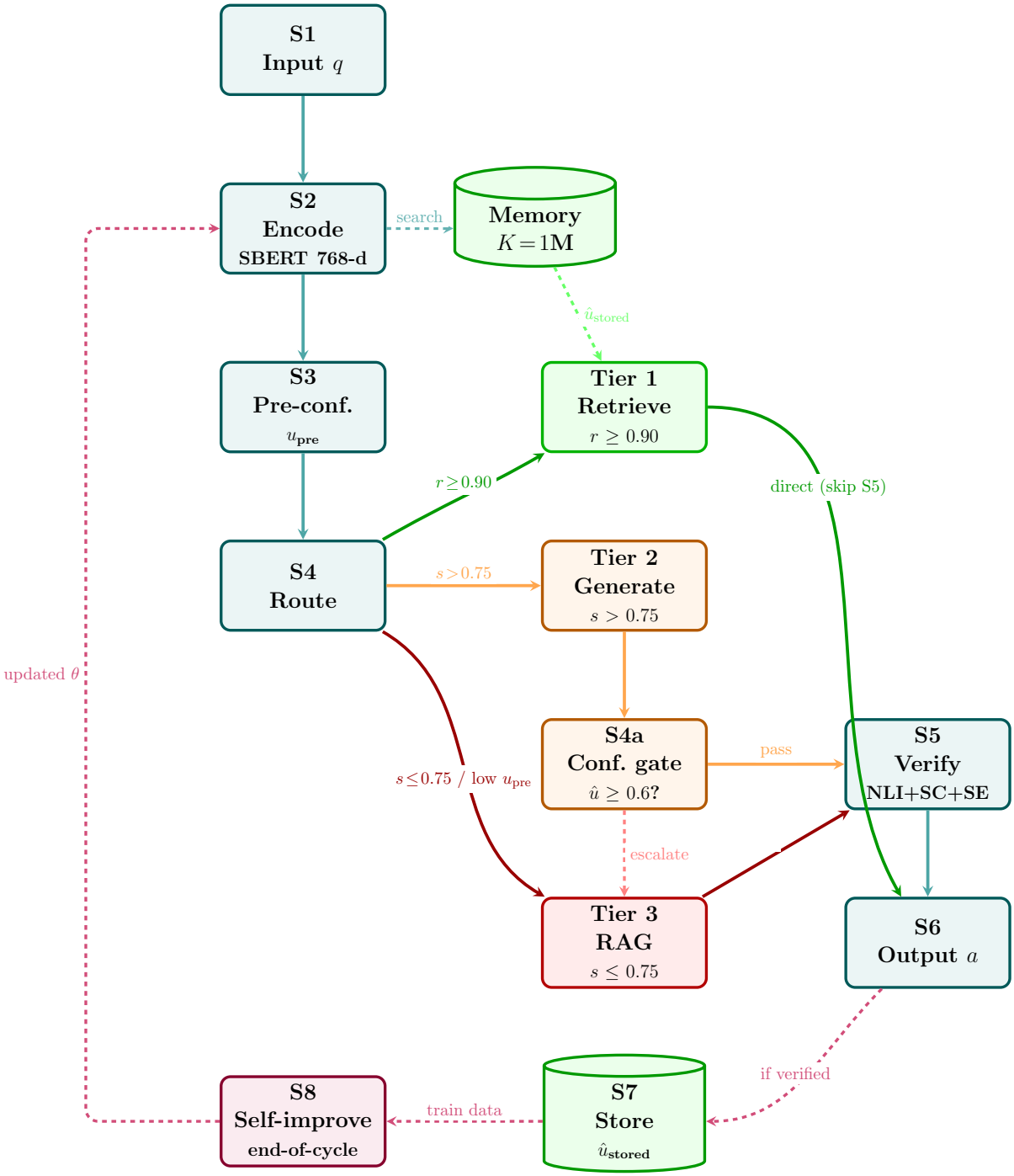


Figure 1.1: CAEM eight-stage pipeline with three-tier routing.

Algorithm 1.1: CAEM per-query processing pipeline (Stages 1–7).

Query q, episodic memory $\mathcal{M}$, model θ, config $\mathcal{C}$ Answer a, updated $\mathcal{M}$
$\phi(q) \leftarrow \text{SBERT}(q)$; $(e^*, s) \leftarrow \text{FAISSSEARCH}(\mathcal{M}, \phi(q))$ Stages 1–2
$u_{\text{pre}} \leftarrow 0.60\, u_{\text{token}}(q) + 0.40\, u_{\text{cconv}}(q)$ Stage 3
$u_{\text{pre}} < \mathcal{C}.\texttt{safety\_u\_pre\_min}$ OR-condition: force Tier 3
$(c, a) \leftarrow \text{RAGGENERATE}(q)$; tier $\leftarrow 3$
$0.7\, s + 0.3\, \hat{u}_{\text{stored}}(e^*) \geq 0.90$ Tier 1: retrieve from memory

---

$(e^*.\texttt{answer},\ \mathcal{M})$ Stages 4a and 5 skipped
$s > 0.75$ Tier 2: generate with CoT $(c,\ a) \leftarrow \theta(q,\ \text{CoT prompt})$
$\hat{u} \leftarrow \textsc{PostGenConf}(q,\ c,\ a)$ Stage 4a; Eq. **??**
$\hat{u} < 0.60$ Escalate to Tier 3 $(c,\ a) \leftarrow \text{RAG}\textsc{Generate}(q)$; tier $\leftarrow 3$
tier $\leftarrow 2$
Tier 3: similarity too low $(c,\ a) \leftarrow \text{RAG}\textsc{Generate}(q)$; tier $\leftarrow 3$
$(\textit{accept},\ \hat{u}_{\text{stored}}) \leftarrow \textsc{Verify}(q,\ c,\ a)$ Stage 5; Alg. **??**
Output a to user Stage 6
accept $\mathcal{M}.\textsc{Store}(q,\ c,\ a,\ \phi(q),\ \hat{u}_{\text{stored}})$ Stage 7: persist verified episode

---

Continual Learning Positioning

CAEM operates in the continual learning paradigm [**?**]: the model is updated across sequential cycles without access to all prior training data simultaneously. Unlike standard continual learning benchmarks that present distinct sequential tasks, CAEM's task sequence is not a change of task but a progressive increase in training data quality on the same tasks. Catastrophic forgetting therefore manifests as degradation of general-domain language capabilities, not as forgetting a prior task. The L2 regularisation mechanism in Stage 8 (Section **??**) addresses this by penalising large deviations from the previous cycle's weights. Chapter 1 provides a higher-level overview of the same architecture (Figure **??**).

1.2 Confidence Estimation and Adaptive Routing

CAEM uses three confidence values that are computed at different pipeline stages and serve different purposes. They must not be confused:

- u_{pre} (*pre-routing confidence*): computed at Stage 3 from encoder-only signals, before any generation. Used exclusively as the OR-condition safety veto in routing.
- $\hat{u}$ (*post-generation confidence*): computed at Stage 4a from four expensive signals, *only for Tier 2 answers*. Used as an efficiency gate before the verification pipeline.
- $\hat{u}_{\text{stored}}$ (*stored episode confidence*): computed at Stage 7 from the verification pipeline's outputs, for all stored episodes regardless of tier. Used in Tier 1 routing score and pruning priority.

These three values are summarised in Table **??**; their signal composition diagrams appear in Figure **??** at the end of this section.

Value	Stage	Formula	Purpose	Compute cost
u_{pre}	3 (pre-gen)	$0.6\,u_{\text{token}} + 0.4\left(\frac{1}{1+C_{\text{conv}}}\right)$	Routing OR veto	Near-zero
$\hat{u}$	4a (post-gen, Tier 2 only)	$\sum w_i \cdot s_i$ (4 signals)	Accept/fallback gate	Expensive
$\hat{u}_{\text{stored}}$	7 (at storage)	$0.5\,p_{\text{entail}} + 0.3\,s_{\text{avg}} + 0.2(1-h_{\text{norm}})$	Tier 1 routing; pruning	Zero at query time

Table 1.2: Three confidence values used in CAEM.

1.2.1 Pre-Routing Confidence Estimation

Before any generation, the system computes u_{pre} using only signals available from the encoding step. Two signals are used at this stage (not four) because generation has not yet occurred; token-level and dropout-based signals from the *answer* are not yet available.

$$u_{\text{cconv}} = \frac{1}{1 + C_{\text{conv}}}, \qquad u_{\text{pre}} = 0.6\,u_{\text{token}} + 0.4\,u_{\text{cconv}} \tag{1.1}$$

where u_{token} is the geometric-mean token log-probability from a single greedy decode pass over the question tokens, and C_{conv} is the convergence coefficient:

$$C_{\text{conv}} = \frac{\text{Var}(h_{1:\lfloor L/2 \rfloor})}{\text{Var}(h_{\lfloor L/2 \rfloor:L}) + \epsilon} \tag{1.2}$$

with h_i the encoder hidden state vectors at layer i, $L = 12$ encoder layers in Flan-T5-Large, and $\epsilon = 10^{-8}$ for numerical stability. In implementation, the raw ratio C_{conv} is transformed into a bounded confidence term $u_{\text{cconv}} = 1/(1 + C_{\text{conv}})$ before combining with u_{token}. This conservative mapping means larger variance-ratio values contribute lower confidence.

Adaptation from prior work. C_{conv} was introduced by Nandakishor [**?**] for decoder-only models. In CAEM, it is applied to Flan-T5-Large's *encoder* hidden states specifically, because the goal is to measure query-understanding confidence *before* generation begins. This adaptation is deliberate and is acknowledged explicitly here.

Hyperparameter provenance. The weights in Equation **??** (0.6/0.4) are design choices [DES]: u_{token} is the primary reliability signal because it directly reflects the model's learned response distribution; the transformed u_{cconv} term is secondary, providing a geometric check on representational stability.

1.2.2 Three-Tier Adaptive Routing

The routing architecture is the mechanism by which CAEM trades computational cost against answer quality. Table **??** summarises the three tiers.

Tier	Condition	Source	Est. latency	Fraction
1	$0.7s + 0.3\hat{u}_{\text{stored}} \geq 0.90$	Episodic memory retrieval	<400 ms	∼50%
2	$s > 0.75$ **AND** $u_{\text{pre}} \geq 0.6$ **AND** $r(q, e) < 0.90$	Fine-tuned model generation	<2.5 s	∼35%
3	$s \leq 0.75$ **OR** $u_{\text{pre}} < 0.6$	RAG + Wikipedia retrieval	<6 s	∼15%

Table 1.3: Three-tier routing summary.

As Tier 1 memory coverage grows across cycles, the average query latency falls: the system becomes computationally cheaper as it becomes more knowledgeable. This "computational dividend" is reported in Chapter 5 (5.6).

Tier 3 retrieval corpus. Tier 3 RAG retrieval uses the full DPR Wikipedia passage corpus [**?**], consisting of 21,015,324 passages derived from the December 2018 Wikipedia dump segmented into non-overlapping 100-word chunks. Passages are indexed with a FAISS IVF-PQ index (65,536 coarse centroids, $M = 64$ bytes per vector, 8 bits per sub-quantiser), occupying approximately 40 GB on disk and requiring at least 128 GB system RAM for full in-memory operation on a cloud A100 node. At query time, the top-k passages (default $k = 5$) are concatenated with the query and passed to the Flan-T5 encoder as additional context. This corpus provides broad factual grounding for queries where neither episodic memory nor standalone model generation is sufficiently reliable.

The routing decision applies two mechanisms sequentially. They are deliberately separated and do not interact mathematically.

Mechanism 1: OR-condition safety veto (checked first).

$$\text{if } u_{\text{pre}} < 0.6 \Rightarrow \text{route to Tier 3, regardless of similarity} \tag{1.3}$$

The OR-condition is a safety-first design principle. Even when the nearest stored episode has high similarity, low pre-routing confidence indicates that the model does not reliably understand the query in its current state. Routing such a query to Tier 2 risks a confident wrong answer. The computational cost of Tier 3 is explicitly preferred over that risk. Separating the veto from the routing score matters: combining them into a single formula would allow a high $\hat{u}_{\text{stored}}$ to arithmetically compensate for a dangerously low u_{pre}, violating the safety-first principle.

Mechanism 2: Routing score (applied only if OR-condition does not fire).

$$r(q, e) = 0.7\, s(q, e) + 0.3\, \hat{u}_{\text{stored}}(e) \tag{1.4}$$

where $s(q, e)$ is the cosine similarity between $\phi(q)$ and the stored episode embedding $\phi(e)$, and $\hat{u}_{\text{stored}}(e)$ is the episode's stored verification confidence. The routing decision is:

$$r(q, e) \geq 0.90 \quad \Rightarrow \quad \text{Tier 1: retrieve from memory} \tag{1.5}$$

$$r(q, e) < 0.90 \text{ AND } s > 0.75 \text{ AND } u_{\text{pre}} \geq 0.6 \quad \Rightarrow \quad \text{Tier 2: generate} \tag{1.6}$$

$$s \leq 0.75 \text{ OR } u_{\text{pre}} < 0.6 \quad \Rightarrow \quad \text{Tier 3: RAG} \tag{1.7}$$

The decision process is formally defined as pseudocode in Algorithm **??**.

Algorithm 1.2: Three-tier routing decision.

[1]

Pre-routing confidence u_{pre}; nearest episode e^* with sim s and $\hat{u}_{\text{stored}}(e^*)$ Routing decision $\in$ {Tier 1, Tier 2, Tier 3} $u_{\text{pre}} < 0.60$ Tier 3 OR-condition: safety override, checked first
$r \leftarrow 0.7\, s + 0.3\, \hat{u}_{\text{stored}}(e^*)$ routing score
$r \geq 0.90$ Tier 1 `tier1_combined_threshold`
$s > 0.75$ Tier 2 `tier2_similarity_threshold`
Tier 3

Hyperparameter provenance. The routing score weights (0.7/0.3) are design choices [DES]: similarity dominates because it is the primary relevance signal; stored confidence corrects at the margin. The Tier 1 threshold (0.90) is a design choice [DES] set conservatively to minimise near-miss retrieval, which is particularly important for FEVER, where adversarially constructed claims share high structural similarity but have opposite labels. The Tier 2 similarity boundary ($s > 0.75$), Tier 1 combined threshold (0.90), and the OR-condition threshold (0.60) are design choices [DES] determining the boundary between medium-confidence and low-confidence routing zones.

Tier 1 skips Stage 5 per query. Tier 1 serves retrieved answers directly from the stored episode without re-running the verification pipeline for each query. Quality is maintained by retroactive re-verification (Section **??**), which re-scores all stored episodes with the improved model once per cycle. This amortises verification cost over one batch per cycle rather than per query, a strictly stronger freshness guarantee than per-query re-verification. Tier 1 latency is therefore dominated by FAISS retrieval only (≈150–400 ms), not by the NLI+SC+SE pipeline.

1.2.3 Post-Generation Confidence Estimation

After Tier 2 generation completes, Stage 4a computes the post-generation composite confidence $\hat{u}$ from four signals. This stage is an *efficiency gate*, not a quality gate: its purpose is to avoid running the expensive verification pipeline on answers that the model itself rates as low-confidence. Stage 5 verification makes the final quality judgment; Stage 4a merely pre-filters obvious failures.

$$\hat{u} = w_1\, u_{\text{token}} + w_2\, u_{\text{dropout}} + w_3\, u_{\text{consistency}} + w_4\, u_{\text{entropy}} \tag{1.8}$$

The four signals are:

- u_{token}: geometric-mean token log-probability over the generated answer (measures token-level generation certainty).
- u_{dropout}: MC Dropout uncertainty, estimated from $K = 5$ stochastic forward passes with dropout rate $p = 0.1$ (Gal & Ghahramani, 2016 [**?**]). A lower variance across passes indicates higher confidence.
- $u_{\text{consistency}}$: average pairwise cosine similarity across $M = 3$ independent self-consistency samples (Wang et al., 2023 [**?**]). High consistency signals that multiple reasoning paths agree.

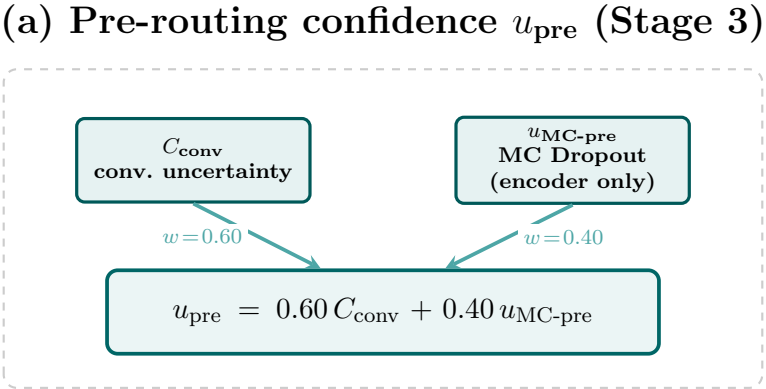
- u_{entropy}: semantic entropy from $K = 10$ samples at temperature $T = 1.0$, clustered by NLI-based meaning equivalence (Farquhar et al., 2024 [**?**]). In code, this signal is represented as a confidence term $u_{\text{entropy}} = 1 - H_{\text{semantic}} / \log_2(K)$, so higher values indicate lower semantic-level uncertainty.

Hyperparameter provenance. The MC Dropout parameters ($K = 5$, $p = 0.1$) are fixed from Gal & Ghahramani (2016) [LIT]. The SC sample count ($M = 3$) follows the implementation default [LIT]. The SE sample count ($K = 10$, $T = 1.0$) and clustering method (agglomerative + cosine NLI) are fixed from Farquhar et al. (2024) [LIT].
Initial signal weights are equal: $w_1 = w_2 = w_3 = w_4 = 0.25$ [CAL initial]. After temperature scaling calibration on the 500-sample calibration set, weights are projected to converge towards $\approx 0.20/0.20/0.20/0.40$, with semantic entropy receiving elevated weight based on its superior AUROC for confabulation detection (AUROC ≈ 0.79, Farquhar et al. 2024). *The 0.20/0.20/0.20/0.40 values are planning projections, not pre-set values.* Actual calibrated weights are reported in Chapter 5's calibration results. If $\hat{u} < 0.6$, the Tier 2 answer is discarded and the query is escalated to Tier 3.
Each signal catches a distinct failure mode. No single signal is sufficient for reliable hallucination detection. Table **??** shows the failure mode each signal addresses and its blind spot.

Signal	**What it catches**	**Blind spot**
u_{token}	Lexical uncertainty; unknown entities	Confidently wrong on memorised misconceptions
u_{dropout}	Epistemic uncertainty; out-of-distribution queries	High variance on genuinely ambiguous (but correct) answers
$u_{\text{consistency}}$	Reasoning instability; multiple incoherent chains	Coherently wrong reasoning that consistently produces the same wrong answer
u_{entropy} (SE)	Systematic misconceptions; confabulation	Diverse but partially correct answers (may over-penalise)

Table 1.4: Verification signal coverage and blind spots.

Figure **??** summarises the signal architecture for both u_{pre} and $\hat{u}$; having introduced both estimates in the preceding subsections, the two-panel layout can now be read as a unified reference.



(b) Post-generation confidence $\hat{u}$ (Stage 4a, Tier 2 only)

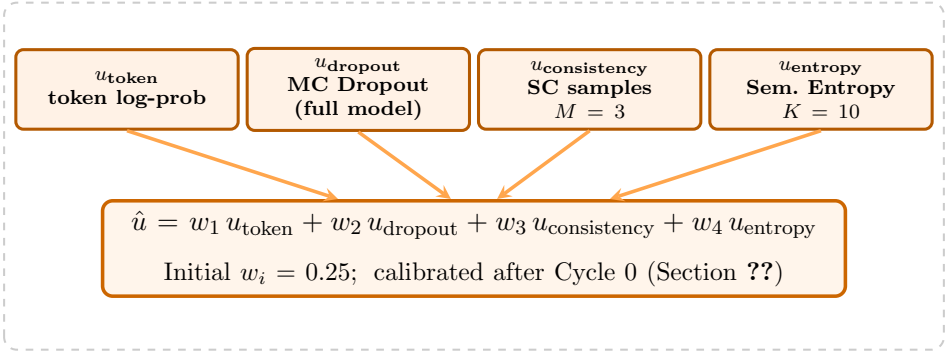


Figure 1.2: Confidence signal architecture: u_{pre} (Stage 3) and $\hat{u}$ (Stage 4a).

1.3 Verification Pipeline and Episodic Memory

1.3.1 Multi-Signal Verification (Stage 5)

All Tier 2 answers that pass Stage 4a, and all Tier 3 answers, enter the Stage 5 verification pipeline. The current implementation combines three verification signals into a continuous stored-confidence score (Equation **??**) without benchmark-specific hard escalation rules.

Layer 1: NLI. RoBERTa-Large-MNLI [**?**] is used to compute entailment probabilities over generated reasoning chains. For each query, $M = 3$ sampled chains are compared against the final answer, and the entailment probability is averaged to obtain p_{entail}.

Layer 2: Self-Consistency (SC). Three independent answer chains are generated at temperature $T = 0.7$. The average pairwise cosine similarity s_{avg} measures agreement across chains. High consistency is a necessary (though not sufficient) signal of reliable generation.

Layer 3: Semantic Entropy (SE). Semantic entropy H_{SE} is computed by unconditionally generating $K = 10$ diverse candidate answers at elevated temperature ($T = 1.0$) [**?**]. These candidates are grouped into semantic equivalence classes using the NLI model. Specifically, agglomerative clustering is applied where two answers are placed in the same cluster G_c if and only if they demonstrate bidirectional entailment. The probability of each cluster is defined proportionally, $p_c = |G_c|/K$, allowing entropy to be computed across distinct meanings as $H_{\text{SE}} = -\sum_c p_c \log p_c$. A high-entropy output distribution indicates that the model is generating diverse meanings: the hallmark of confabulation rather than genuine uncertainty about phrasing.

Implementation note. Configuration placeholders exist for additional VE-style escalation heuristics, but the active verifier path currently uses the continuous three-signal composite score in Equation **??**.

Task-aware label extraction. Tier 2 and Tier 3 generation uses task-aware prompts that produce structured reasoning traces for all three training benchmarks (FEVER, TriviaQA, Natural Questions). For classification benchmarks (FEVER), the generated output has the form "`Reasoning: <explanation>`" followed by "`Answer: <label>`". At evaluation time, a dedicated extraction function (`extract_fever_label()`) uses regular-expression matching to parse the label from the rationale-style output before scoring. This ensures that the `reasoning_chain` field stored in episodic memory always contains a genuine reasoning trace, consistent with the verified reasoning-chain supervision claim

(Section **??**).

Stored confidence formula. If the answer passes verification, the episode is stored with continuous quality score:

$$\hat{u}_{\text{stored}} = 0.5\, p_{\text{entail}} + 0.3\, s_{\text{avg}} + 0.2\,(1 - h_{\text{norm}}) \tag{1.9}$$

where $p_{\text{entail}} = P(\text{ENTAIL})$ is the NLI entailment probability, s_{avg} is the mean pairwise SC cosine similarity, and $h_{\text{norm}} = H_{\text{SE}}/\log_2 K$ is the normalised semantic entropy.

Critical distinction: $\hat{u}_{\textbf{stored}} \neq \hat{u}$**.** $\hat{u}_{\text{stored}}$ is derived exclusively from the Stage 5 verification pipeline, not from Stage 4a. This is necessary because Tier 3 episodes never execute Stage 4a (they skip the post-generation gate), so they never produce a $\hat{u}$ value. Yet Tier 3 episodes must receive a $\hat{u}_{\text{stored}}$ quality score for Tier 1 routing and pruning priority. The verification pipeline is the only stage shared by all three tiers and is therefore the unique source of $\hat{u}_{\text{stored}}$.

Hallucination rate definition. Following the operational definition established in implementation (see `caem-implementation-log.md`, IMPL-02), hallucination rate is measured as:

$$H = \frac{|\{q : \text{EM}(q) = 0 \ \wedge \ \hat{u}_{\text{stored}}(q) \geq 0.5\}|}{N}$$

This captures *confident confabulation*, the most harmful failure mode. Low-confidence or verification-rejected wrong answers do not satisfy this condition; the metric focuses on wrong answers that remain high-confidence after the storage-quality gate.

The verification pipeline is formally defined as pseudocode in Algorithm **??**.

Algorithm 1.3: Multi-signal verification pipeline (Stage 5)

Query q, reasoning chain c, answer a (*accept* $\in \{\text{T}, \text{F}\}$, $\hat{u}_{\text{stored}} \in [0, 1]$) **Layer 1: NLI Entailment**
Sample $M = 3$ chains $\{c_j\}$ at temperature $T = 0.7$
each chain c_j $p_j \leftarrow$ RoBERTa-MNLI$(c_j \Rightarrow a)$
$p_{\text{entail}} \leftarrow \frac{1}{M}\sum_j p_j$
Layer 2: Self-Consistency $s_{\text{avg}} \leftarrow$ mean pairwise cosine similarity of $\{c_j\}$ reuse chains from Layer 1
Layer 3: Semantic Entropy Sample $K = 10$ answers $\{a_k\}$ at $T = 1.0$
Cluster $\{a_k\}$ by NLI-based semantic equivalence (agglomerative)
$h_{\text{norm}} \leftarrow H_{\text{SE}}/\log_2 K, \quad H_{\text{SE}} = -\sum_c P(c) \log P(c)$
Combine and Gate $\hat{u}_{\text{stored}} \leftarrow 0.5\, p_{\text{entail}} + 0.3\, s_{\text{avg}} + 0.2\,(1 - h_{\text{norm}})$ Eq. **??**
$\hat{u}_{\text{stored}} \geq 0.50$ (True, $\hat{u}_{\text{stored}}$)
(False, $\hat{u}_{\text{stored}}$)

1.3.2 Episodic Memory Architecture

Each stored episode is a structured record with two classes of fields: *immutable* content fields and *mutable* quality fields.

Field	Type	Description
Immutable content fields (set at storage, never updated)		
`question`	str	Original query q
`context`	str	Retrieved passage (Tier 2/3) or empty (Tier 1)
`reasoning_chain`	str	Full task-conditioned reasoning trace + answer
`answer`	str	Final answer string (extracted from reasoning chain)
`embedding`	float[768]	SBERT embedding $\phi(q)$
`t_cycle`	int	Cycle index at storage time
Mutable quality fields (updated across cycles)		
`u_stored`	float	Stored confidence $\hat{u}_{\text{stored}} \in [0, 1]$; Eq. **??**
`p_entail`	float	NLI entailment probability
`s_avg`	float	Mean pairwise SC cosine similarity
`h_norm`	float	Normalised semantic entropy
`retrieval_count`	int	Number of times served at Tier 1
`success_rate`	float	Fraction of retrievals judged correct
`retroverified`	bool	Whether re-scored by retroactive re-verification

Table 1.5: Episodic memory episode schema.

Why content fields are immutable. The fine-tuning dataset for Stage 8 is derived directly from stored reasoning chains. If content fields were updated mid-cycle, the model would be trained on a moving target, violating the clean generate/improve separation that the verified rejection sampling framework requires. Immutable content ensures that the fine-tuning dataset is auditable: every training example can be traced back to the specific query, context, and reasoning chain that the verifier accepted.

Pruning policy. When memory approaches capacity (K = 1,000,000 episodes), pruning triggers at 95% occupancy (950,000 episodes). Episodes are pruned by ascending $\hat{u}_{\text{stored}} \times$ success_rate: the lowest combined quality episodes are removed first. This preserves the highest-confidence and most-frequently-retrieved knowledge in memory. At the chosen scale, the FAISS IVF-PQ index (4,096 coarse centroids, $M = 64$, 8 bits per sub-quantiser) occupies approximately 70 MB in memory, well within GPU VRAM alongside both language models.

1.4 Self-Improvement Loop and Calibration

1.4.1 Iterative Self-Improvement (Stage 8)

Stage 8 implements *verified rejection sampling*: generated answers are accepted only if they pass the multi-signal verification pipeline, and only accepted samples are used for fine-tuning. This is analogous to Reinforced Self-Training (ReST, Gulcehre et al., 2023 [**?**]), which alternates between a *grow* step (sampling high-reward outputs) and an *improve* step (fine-tuning on those outputs). CAEM's specific contributions over ReST are

three-fold: (1) multi-signal verification (NLI + SC + SE) replaces a scalar reward model, catching distinct hallucination failure modes that a single score cannot separate; (2) episodic memory enables routing-based reuse of verified knowledge across cycles, providing an inference-time dividend that ReST does not offer; and (3) retroactive re-verification propagates model improvements backwards into memory quality.
The ten-cycle structure is illustrated in Figure **??**.

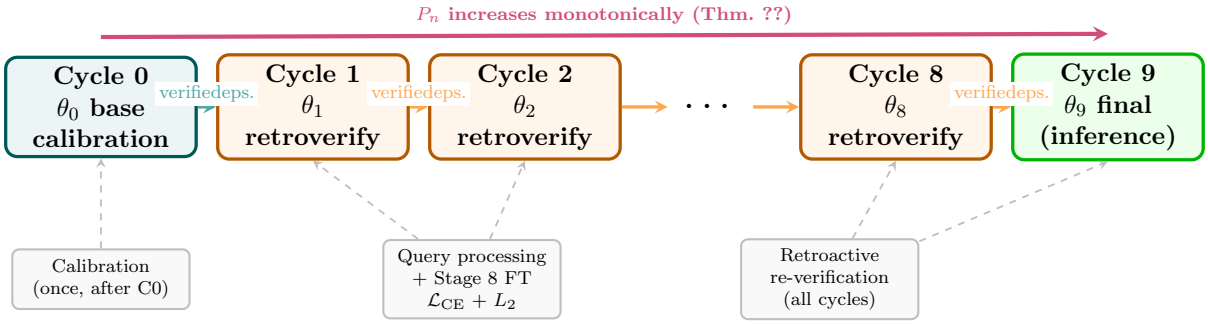


Figure 1.3: CAEM ten-cycle self-improvement structure (C0–C9). Calibration of $\hat{u}$ signal weights runs once after Cycle 0 (Section **??**). Each subsequent cycle fine-tunes on freshly verified episodes and runs retroactive re-verification to refresh memory quality scores. Memory purity P_n increases monotonically across all cycles (Theorem **??**, Section **??**). The final weights θ_9 are used for inference after Cycle 9.

Training objective. The fine-tuning objective trains on the full reasoning chain:

$$\mathcal{L}_{\text{data}} = -\sum_{(q,c,a)\in\mathcal{D}_{\text{verified}}} \log P_\theta(c, a \mid q) \tag{1.10}$$

where c is the task-conditioned reasoning trace and a is the extracted answer. Training on the full reasoning chain (not isolated answer strings) provides richer sequence-level supervision and is consistent with chain-of-thought distillation literature [**?**, **?**].
Task-aware prompts. All three training benchmarks use structured generation prompts that ensure `reasoning_chain` always contains genuine reasoning content:

- **Classification** (FEVER):

```
Determine whether the claim [supports/refutes/not enough
info].
Format: Reasoning: <explanation>
Answer: <label>
```

- **Open-ended** (TriviaQA):

```
Question: {q}
Think step by step:
Answer:
```

- **Open-ended** (Natural Questions):

```
Question: {q}
Think step by step:
Answer:
```

Transfer evaluation benchmarks (TruthfulQA, StrategyQA, ARC-Challenge) use the same open-ended or classification templates respectively, but their samples are never used for self-improvement training; they appear only in held-out evaluation.

Stability-plasticity tradeoff. The model must be plastic enough to absorb new verified knowledge each cycle, yet stable enough to retain general-purpose language capabilities. Without regularisation, each fine-tuning cycle would overwrite previously learned weights, a phenomenon known as catastrophic forgetting [**?**]. CAEM addresses this via **L2 regularisation with uniform parameter weighting**, adding a penalty to the standard cross-entropy loss:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \frac{\lambda}{2} \|\theta - \theta_{\text{prev}}\|^2 \tag{1.11}$$

where θ_{prev} are the weights from the end of the previous cycle and $\lambda = 0.01$ [DES].

Full Elastic Weight Consolidation (EWC, Kirkpatrick et al., 2017 [**?**]) weights this penalty per-parameter by the Fisher Information Matrix (FIM), concentrating regularisation on parameters most critical to prior performance. CAEM uses the uniform-weight approximation (FIM $\approx$ uniform), which is computationally efficient and appropriate for diverse QA fine-tuning where the gradient signal distributes broadly across parameters. The validity of this approximation is empirically supported by the MMLU retention results (Chapter 5, §5.5.2) and by the per-cycle relative retention guard used during training: if the approximation were substantially incorrect, retention would fail the 93% threshold.

Forgetting abort guard. An independent forgetting abort guard prevents runaway degradation. After each cycle's fine-tuning, the system computes:

$$\rho = \frac{\text{General-domain accuracy (post-tuning)}}{\text{General-domain accuracy (pre-tuning)}}$$

In implementation, this general-domain score is measured on a held-out general-QA subset (TriviaQA-derived proxy) and compared as a relative ratio. If $\rho < 0.93$ (relative retention ratio falls below 93%), fine-tuning weights for that cycle are discarded and θ_{prev} is restored. This hard safety net operates independently of λ calibration.

Algorithm 1.4: CAEM cyclic self-improvement (Stage 8).

---

[1]

Verified episode set $\mathcal{D}_{\text{verified}}$; general QA pool $\mathcal{D}_{\text{general}}$; θ_{prev}; $\lambda = 0.01$; $\rho_{\min} = 0.93$ Updated weights θ (or θ_{prev} if forgetting abort fires)
Split $\mathcal{D}_{\text{general}}$ into $\mathcal{D}_{\text{gen-train}}$ and held-out $\mathcal{D}_{\text{gen-eval}}$
$\mathcal{D}_{\text{train}} \leftarrow \textsc{Mix}(\mathcal{D}_{\text{verified}}, \mathcal{D}_{\text{gen-train}};\ 90\%/10\%)$
$A_{\text{pre}} \leftarrow \textsc{Evaluate}(\theta_{\text{prev}}, \mathcal{D}_{\text{gen-eval}})$ pre-tuning retention baseline
$\theta \leftarrow \theta_{\text{prev}}$
each epoch in fine-tuning schedule each batch $(q, c, a) \in \mathcal{D}_{\text{train}}$
$\mathcal{L} \leftarrow -\log P_\theta(c, a \mid q) + \frac{\lambda}{2}\|\theta - \theta_{\text{prev}}\|^2$
Update θ via AdamW on $\mathcal{L}$; clip gradient norm to 1.0
$A_{\text{post}} \leftarrow \textsc{Evaluate}(\theta, \mathcal{D}_{\text{gen-eval}})$ post-tuning retention
$A_{\text{post}}/A_{\text{pre}} < \rho_{\min}$ catastrophic forgetting θ_{prev} abort: restore previous cycle's weights
$\theta_{\text{prev}} \leftarrow \theta$
θ

1.4.2 Retroactive Re-verification

At the end of each cycle, after Stage 8 fine-tuning, the improved model re-scores all existing episodes in memory. For each stored episode, the verification pipeline (Section **??**) is re-run using the current model's outputs, and the mutable quality fields ($\hat{u}_{\text{stored}}$, p_{entail}, s_{avg}, h_{norm}) are updated accordingly. Episodes that no longer pass verification under the improved verifier are flagged or pruned.

This mechanism provides two guarantees. First, *freshness*: episodes stored in early cycles with a weaker verifier are retrospectively assessed by the stronger verifier, preventing stale low-quality episodes from persisting in Tier 1. Second, *upward pressure on memory quality*: as $\hat{u}_{\text{stored}}$ scores are updated to reflect the improved verifier, the average $\hat{u}_{\text{stored}}$ in memory rises monotonically, a consequence of Theorem **??** (Chapter 4, §**??**).

Tier 1 quality assurance is therefore maintained without per-query re-verification cost. The retroactive batch adds one verification pass per cycle (not per query), which is substantially cheaper than running Stage 5 at every Tier 1 retrieval. Algorithm **??** formalises the procedure.

Algorithm 1.5: Retroactive re-verification (end of cycle).

---

[1]

Memory $\mathcal{M}$; updated model θ; eviction threshold $\tau = 0.50$ Updated $\mathcal{M}$ with refreshed quality scores each episode $e \in \mathcal{M}$ $(ok, \hat{u}'_{\text{stored}}) \leftarrow \textsc{Verify}(\theta,\ e.\texttt{question},\ e.\texttt{reasoning\_chain},\ e.\texttt{answer})$ $\hat{u}'_{\text{stored}} < \tau$ Remove e from $\mathcal{M}$ quality eviction $e.\texttt{u\_stored} \leftarrow \hat{u}'_{\text{stored}}$; $e.\texttt{retroverified} \leftarrow$ True $\mathcal{M}$

1.4.3 Confidence Calibration

Before Cycle 1 fine-tuning begins, the base model (Cycle 0) undergoes a one-time calibration step. Modern neural networks are often overconfident: their predicted confidence scores substantially exceed actual accuracy [**?**]. Calibration corrects this miscalibration so that $\hat{u}$ scores reliably reflect true answer quality.

Temperature scaling. A single learnable scalar T is fitted by minimising Expected Calibration Error (ECE) on a 500-sample held-out calibration set using L-BFGS:

$$p'_\theta(y \mid x) = \text{softmax}\Big(\frac{z}{T}\Big) \tag{1.12}$$

where z is the logit vector and $T > 1$ softens the distribution (reduces overconfidence). ECE is formally defined as:

$$\text{ECE} = \sum_{b=1}^{M} \frac{|B_b|}{N} \big|\, \text{acc}(B_b) - \text{conf}(B_b) \big| \tag{1.13}$$

where $M = 10$ bins, B_b is the set of predictions with confidence in bin b, $\text{acc}(B_b)$ is the fraction correct in that bin, and $\text{conf}(B_b)$ is the mean predicted confidence. A perfectly

calibrated model has ECE = 0. Temperature scaling is the most parameter-efficient calibration method and achieves competitive ECE reduction without degrading accuracy [**?**].

Signal weight calibration. The $\hat{u}$ signal weights $(w_1, \ldots, w_4)$ are initialised at 0.25 (equal) and fitted via logistic regression on the same 500-sample calibration set, maximising AUROC for the binary correct/incorrect classification. Semantic entropy is initialised at elevated weight (approximately 0.40) consistent with its empirically demonstrated AUROC advantage [**?**]; the calibration may adjust this.

Timing. Calibration is applied *once*, after Cycle 0 inference and before Cycle 1 fine-tuning. The calibrated T value and signal weights are fixed for all subsequent cycles. The 500-sample calibration set is drawn from held-out portions of each benchmark's validation split and is *separate* from the 500-sample purity validation set used for theory validation in Chapter 5.

Reported values. The fitted T value and the actual calibrated signal weights are reported in Chapter 5 (§5.1.3), generated through the automated calibration suite after the full experiment completes. The 0.20/0.20/0.20/0.40 projection appearing in this chapter is a planning estimate only; Chapter 5 reports the actual measurements.

1.5 Theoretical Analysis

This section provides formal theoretical foundations for CAEM's self-improvement mechanism. Three results are established: a data purity theorem showing that verified episodic memory is more reliable than the model that produced it; a recurrence monotonicity theorem showing that purer memory produces a better model; and a convergence bound showing that improvement is monotone but bounded by a practical ceiling.

All statements in this section use symbolic variables only. Actual empirical values (measured p, α, and P_{obs} from the experiment) appear in Chapter 5, within the theory-validation results.

1.5.1 Notation and Definitions

Definition 1.1 (Model accuracy p)**.** Let $p \in (0, 1)$ denote the probability that the model generates a correct answer on a uniformly drawn query. Correctness is measured by exact match against the ground-truth answer.

Definition 1.2 (Verifier accuracy α)**.** Let $\alpha \in (0, 1)$ denote the accuracy of the multi-layer verification pipeline, measured as:

$$\alpha = \frac{\text{TP} + \text{TN}}{N_{\text{purity}}} \tag{1.14}$$

where N_{purity} is the size of the purity validation set, TP is the count of correct answers that the verifier accepts (correct accept), and TN is the count of incorrect answers that the verifier rejects (correct reject). A false positive (FP) is a stored hallucination: the model is wrong but the verifier accepts. A false negative (FN) is a correct answer that the verifier unnecessarily rejects.

Remark 1.3. Under Definition **??**, the probability of accepting a correct answer is α and the probability of accepting an incorrect answer is $1 - \alpha$. This symmetric treatment holds

when the pipeline is tuned to balance precision and recall, which is the case in CAEM's design (see Section **??**).

Definition 1.4 (Memory purity P)**.** Let $P \in (0,1)$ denote the fraction of stored episodes whose answers are correct:

$$P = P(\text{answer correct} \mid \text{verifier accepts})$$

1.5.2 Theorem 1: Data Purity

Theorem 1.5 (Data Purity)**.** *Under Definitions 4.1–4.3, the memory purity satisfies:*

$$P = \frac{p\alpha}{p\alpha + (1-p)(1-\alpha)} \tag{1.15}$$

Moreover, $P > p$ if and only if $\alpha > \dfrac{1}{2}$.

Proof. We model each generated answer as independently correct with probability p and incorrect with probability $1-p$. By Definition **??**, the verification pipeline accepts a correct answer with probability α and accepts an incorrect answer with probability $1-\alpha$. *Deriving the purity formula.* By the law of total probability, the marginal probability of acceptance is:

$$\begin{aligned} P(\text{accepted}) &= P(\text{accepted} \mid \text{correct}) \cdot P(\text{correct}) \\ &\quad + P(\text{accepted} \mid \text{incorrect}) \cdot P(\text{incorrect}) \\ &= \alpha p + (1-\alpha)(1-p). \end{aligned} \tag{1.16}$$

Applying Bayes' theorem:

$$\begin{aligned} P &= P(\text{correct} \mid \text{accepted}) \\ &= \frac{P(\text{accepted} \mid \text{correct}) \cdot P(\text{correct})}{P(\text{accepted})} \\ &= \frac{\alpha p}{\alpha p + (1-\alpha)(1-p)}, \end{aligned} \tag{1.17}$$

which establishes (**??**).
Proving $P > p \Leftrightarrow \alpha > \frac{1}{2}$. Starting from $P > p$ with $p \in (0,1)$ and the denominator of (**??**) positive:

$$\begin{aligned} \frac{\alpha p}{\alpha p + (1-\alpha)(1-p)} &> p \\ \alpha p &> p\big[\alpha p + (1-\alpha)(1-p)\big] \\ \alpha &> \alpha p + (1-\alpha)(1-p) && \text{(dividing by } p > 0\text{)} \\ \alpha &> \alpha p + 1 - \alpha - p + \alpha p \\ 2\alpha &> 2\alpha p + 1 - p \\ 2\alpha(1-p) &> 1 - p. \end{aligned}$$

Since $p < 1$, we have $(1-p) > 0$ and may divide both sides by $(1-p)$:

$$2\alpha > 1 \quad \Longleftrightarrow \quad \alpha > \tfrac{1}{2}. \qquad \square$$

$\square$

Corollary 1.6 (Useful lower bound)**.** *When $\alpha > \frac{1}{2}$ and $p \geq \frac{1}{2}$, memory purity satisfies $P \geq \alpha$.*

Remark 1.7 (Illustrative example, for pedagogy only). To build intuition, suppose $p = 0.70$ (the model generates correct answers 70% of the time) and $\alpha = 0.85$ (the verifier is correct 85% of the time). Substituting into (**??**):

$$P = \frac{0.70 \times 0.85}{0.70 \times 0.85 + 0.30 \times 0.15} = \frac{0.595}{0.595 + 0.045} \approx 0.929.$$

The stored memory is 92.9% correct, compared to the model's 70% base accuracy. **These numbers are illustrative only.** Actual measured values replace them in Chapter 5 in the theory-validation results subsection.

1.5.3 Theorem 2: Monotone Memory Improvement

Theorem 1.8 (Recurrence Monotonicity)**.** *Let $P(p, \alpha)$ denote the memory purity as a function of model accuracy p for fixed verifier accuracy $\alpha > \frac{1}{2}$. Then $P(p, \alpha)$ is strictly increasing in p. Consequently, if the fine-tuning step at cycle t improves model accuracy from p_t to $p_{t+1} > p_t$, then $P_{t+1} > P_t$.*

Proof. Differentiating (**??**) with respect to p:

$$\frac{\partial P}{\partial p} = \frac{\alpha\big[\alpha p + (1-\alpha)(1-p)\big] - \alpha p\big[\alpha - (1-\alpha)\big]}{\big[\alpha p + (1-\alpha)(1-p)\big]^2}. \tag{1.18}$$

Expanding the numerator:

$$\begin{aligned}
&\alpha\big[\alpha p + 1 - \alpha - p + \alpha p\big] - \alpha p(2\alpha - 1) \\
&= \alpha(2\alpha p + 1 - \alpha - p) - \alpha p(2\alpha - 1) \\
&= \alpha\big[2\alpha p + 1 - \alpha - p - 2\alpha p + p\big] \\
&= \alpha(1 - \alpha).
\end{aligned}$$

Since $\alpha \in (0, 1)$, we have $\alpha(1 - \alpha) > 0$ and the denominator of (**??**) is strictly positive. Therefore $\partial P / \partial p > 0$, confirming P is strictly increasing in p. The corollary $P_{t+1} > P_t$ when $p_{t+1} > p_t$ follows immediately. □ □

Remark 1.9 (Coupling of the two improvement loops). Theorem **??** establishes that the two improvement mechanisms are positively coupled: a better model produces purer memory, and purer memory (via higher-quality fine-tuning data) produces a better model. This creates a virtuous cycle. However, the cycle is bounded, not explosive, because $P < 1$ whenever $\alpha < 1$. The two loops converge rather than diverge; Section **??** formalises the ceiling.

1.5.4 Theorem 3: Convergence and the Practical Ceiling

Theorem 1.10 (Convergence and Practical Ceiling)**.** *Define the per-cycle purity gain as:*

$$\Delta P(p, \alpha) = P(p, \alpha) - p = \frac{p(1-p)(2\alpha - 1)}{\alpha p + (1-\alpha)(1-p)}. \tag{1.19}$$

The following hold:

(a) $\Delta P > 0$ *if and only if* $\alpha > \frac{1}{2}$ *(from Theorem* **??***).*

(b) $\Delta P \to 0$ *as* $p \to 1$ *(diminishing returns as accuracy approaches 1).*

(c) ΔP *is maximised at* $p = p^* = \frac{\sqrt{\alpha(1-\alpha)}-(1-\alpha)}{2\alpha-1}$ *with maximum value* $\Delta P^* = \dfrac{1-2\sqrt{\alpha(1-\alpha)}}{2\alpha-1}$.

(d) The sequence $\{p_t\}$ *is monotonically non-decreasing and bounded above by 1; hence it converges.*

Proof. Part (a) follows directly from Theorem **??**.
Part (b). As $p \to 1$, the numerator $p(1-p)(2\alpha-1) \to 0$ while the denominator $\to \alpha > 0$. Therefore $\Delta P \to 0$.
Part (c). Setting $\partial(\Delta P)/\partial p = 0$ leads to the quadratic equation $(2\alpha-1)p^2 + 2(1-\alpha)p - (1-\alpha) = 0$. Solving for $p \in (0,1)$ yields:

$$p^* = \frac{\sqrt{\alpha(1-\alpha)} - (1-\alpha)}{2\alpha - 1}.$$

Substituting p^* back into the expression for ΔP simplifies to the exact maximum value:

$$\Delta P^* = \frac{1 - 2\sqrt{\alpha(1-\alpha)}}{2\alpha - 1}.$$

Part (d). $\{p_t\}$ is non-decreasing by Theorem **??** and bounded above by 1. By the Monotone Convergence Theorem, the sequence converges to some $p^\infty \leq 1$. □ □

Remark 1.11 (Scoping the convergence claim). Theorem **??** establishes that improvement is *monotone and bounded*, not that it reaches zero hallucination. CAEM does not claim to eliminate hallucination. The practical ceiling is jointly determined by the base model's capacity (p) and the verification pipeline's accuracy (α): even a perfect verifier ($\alpha = 1$) cannot improve accuracy beyond what the model can generate given its parameters. Empirically, the signal that the system is approaching its ceiling is diminishing improvement magnitude between consecutive cycles, confirmed in Chapter 5 by comparing Δ_{avg} across successive cycle pairs.

Remark 1.12 (Practical significance for a 3-cycle experiment). Theorem **??**(c) shows that the maximum per-cycle purity gain occurs at a specific accuracy $p^* < 0.5$. For $\alpha = 0.85$, the maximum gain is $\Delta P^* \approx 0.408$, which occurs exactly at $p^* \approx 0.296$. For a model starting with a base accuracy of $p_0 \approx 0.60$, the system operates to the right of the peak ($p_0 > p^*$). In this region, ΔP is strictly monotonically decreasing as p approaches 1. Therefore, the first cycle captures the largest gain, with subsequent cycles capturing progressively smaller gains as accuracy improves. Three cycles therefore provide a practical approximation of the converging trajectory, capturing the dominant improvement while staying within the computational budget.